

Explain what Functional Reactive Programming Thinking is in contrast to Thinking in Streams, is FRP Thinking closer to Rxjs as Thinking in Streams

Functional Reactive Programming (FRP) Thinking and **Thinking in Streams** are related but distinct mindsets within reactive programming. They both deal with data that changes over time (events, user inputs, async responses), but they differ in core abstractions, approach to time, declarativeness, and how you structure your code. "Thinking in Streams" is the everyday practical approach used in libraries like RxJS, while true **FRP Thinking** (as originally defined) is a more fundamental, mathematically precise paradigm.

Thinking in Streams (the RxJS-style mindset)

This is the dominant way most developers "think reactively" in JavaScript/TypeScript, Java, etc. You model *everything* as discrete sequences of values arriving over time:

Sources (mouse clicks, HTTP responses, timers, WebSocket messages) become **Observables** (streams).

You build **pipelines** by chaining pure functional operators: `map`, `filter`, `merge`, `combineLatest`, `switchMap`, `debounceTime`, etc.

Data "flows" forward (push-based). You subscribe at the end to react to the final stream.

Core mental model:

"Everything is a stream of events/values. I declare *how* the data transforms as it flows from source to consumer."

Example (RxJS):

```
JavaScript

fromEvent(button, 'click')           // stream of click events
  .pipe(
    throttleTime(300),               // operator
    map(() => fetchData()),           // transform
    switchMap(response => response)   // flatten
  )
  .subscribe(data => updateUI(data));
```

This is declarative *within* the pipeline but operational overall: you explicitly create, transform, and subscribe to discrete sequences. It's excellent for handling async complexity, side effects, backpressure, and error handling in real apps. Many tutorials and books explicitly call this "thinking in streams" or "thinking in reactive."

Functional Reactive Programming (FRP) Thinking (the original/purist view)

True FRP was coined by Conal Elliott in the 1990s (with Paul Hudak) in the context of Functional Reactive Animation (Fran). The mindset is fundamentally different:

You work with two first-class abstractions:

Behaviors (or **signals**): *continuous*, time-varying *values* (e.g., mouse position as a function of time, not a stream of discrete updates).

Events: discrete occurrences that can trigger changes.

You define *what* a value *is* declaratively, as a mathematical relationship (like equations). The system automatically computes and propagates updates.

Emphasis on **denotational semantics** (precise mathematical meaning), purity, continuous time, and glitch-free updates. No explicit "listening" or subscriptions in the core model.

Core mental model:

"I describe relationships between time-varying values and events algebraically. The runtime handles *when* and *how* updates happen."

Classic FRP example (pseudocode, as in early Haskell FRP):

Haskell

```
mousePos :: Behavior Point      -- continuous value over time
clicks   :: Event ()            -- discrete events

positionAfterClick :: Behavior Point
positionAfterClick = mousePos `switch` (clicks `then` mousePos)
-- "position is mousePos, but after each click it becomes the new mousePos"
```

It's *not primarily about streams*. Conal Elliott himself has emphasized: "it's certainly not about streams... it's continuous time... streams are a discrete model."

Modern approximations of this thinking appear in **signals** (SolidJS, Svelte runes, Vue 3 refs/computed, Qwik, etc.), where you define reactive dependencies declaratively without manual stream pipelines.

Key contrasts

Aspect	Thinking in Streams (RxJS)	FRP Thinking (original)
Time model	Discrete sequences of values/events	Continuous time-varying behaviors + discrete events
Abstraction	Observables / streams / pipelines	Behaviors (signals) + Events
Style	Compositional operators on flows	Algebraic/denotational definitions of values
Mental focus	"How does data flow and transform?"	"What is this value over time?"
Propagation	Explicit subscribe + push-based	Automatic, glitch-free (pull or push)
Purity	Functional operators, but side effects common in subscriptions	Strongly emphasizes purity and referential transparency
Common use	Async handling, UI events, state management in large apps	Animation, robotics, UI with automatic dependency graphs

Is FRP Thinking closer to RxJS than "Thinking in Streams" is?

No — **Thinking in Streams is the RxJS mindset**. RxJS (and Reactive Extensions in general) is an implementation of **reactive programming** using functional operators on discrete streams. It is *often loosely called "FRP"* in industry tutorials, books, and documentation because it combines **functional programming** with reactivity.

However, according to the original inventor (Conal Elliott) and many purists, **RxJS is not true FRP** — it's discrete, operational, and stream-oriented, whereas classic FRP is continuous-time and denotational. Erik Meijer (Rx creator) has acknowledged this distinction. [News.ycombinator](#)

In short:

If you're using RxJS → you're doing **Thinking in Streams** (very practical, widely used).

If you're using signals or classic FRP systems → you're closer to **true FRP Thinking** (more declarative, "what is the value?" style). Many modern frameworks have converged on signal-based reactivity because it feels closer to original FRP ideals while staying performant in practice. Both mindsets are powerful; the choice depends on whether you need fine-grained control over discrete events (streams) or elegant, automatic time-varying relationships (FRP).

If you're coming from RxJS and want to explore pure FRP Thinking, look at libraries like Reflex (Haskell), Sodium, or modern signal libraries — they shift the mental model from pipelines to continuous behaviors.

to my feeling FRP does more imply Time whereas Stream implies Dataflow but does somehow hide that Time is part of each emitted value: [{Time, Value}]

Yes — your feeling is spot-on and captures one of the deepest, most useful distinctions between the two mindsets.

FRP Thinking = Time is *first-class and explicit*

In true/original Functional Reactive Programming (the Conal Elliott style), you are literally programming with time:

A **Behavior** (or Signal) is defined as a mathematical function:

`Behavior a = Time -> a`

The value *at any point in time* is what matters. Time is not hidden — it is the coordinate system everything lives in.

You describe *relationships* algebraically, and the system automatically keeps everything consistent as time advances.

Events are still discrete, but they mostly serve to *switch* or *sample* behaviors.

So FRP Thinking feels “time-y” because you are constantly answering:

“What *is* this value *right now*, as a continuous function of time?”

Streams (RxJS-style) = Dataflow with *implicit* time

Thinking in Streams is primarily about **data flowing through a graph of transformations**. The mental model is:

An Observable is a **sequence** of discrete values pushed over time.

You compose *dataflow pipelines*: `source.pipe(op1, op2, op3)`.

The timing of each emission is real (wall-clock or scheduler time), but the **API** *hides* it.

You are right that underneath the hood every emission is secretly something like:

TypeScript

```
{ value: T, time: number } // or more precisely a notification at a moment in time
```

RxJS just doesn’t expose that pair directly in the normal API (except via the `timestamp()` operator or virtual-time schedulers in tests). The time dimension is there, but it is *implicit* and *discrete*. You reason about it indirectly through operators like

`debounceTime`, `delay`, `throttleTime`, `bufferTime`, `windowTime`, etc.

So Streams feel like pure **dataflow** because:

Your brain is focused on “how does this piece of data transform and move to the next stage?”

Time only appears when you need to control *when* things happen (ordering, throttling, racing, etc.).

Side-by-side contrast with your intuition

Aspect	FRP Thinking	Streams Thinking (RxJS)
Core question you ask	“What is the value <i>as a function of time</i> ?”	“How does data <i>flow and transform</i> ?”
Time	Explicit, continuous, first-class	Implicit, discrete, hidden behind the sequence
Mental representation	<code>value(t)</code> for any <code>t</code>	Sequence of <code>(value, implicit-timestamp)</code>
How you compose	Algebraic equations on behaviors/events	Pipelines of operators on discrete events
Feels like...	Physics / animation / continuous math	Data pipelines / ETL / event processing

Practical example

FRP (signal/behavior style – pseudo-code):

Haskell

```
mouseX :: Behavior Double      -- continuous value over time
clicks  :: Event ()            -- discrete

-- "the color is red while the mouse is moving right, blue otherwise"
color = mouseX `switch` (clicks `then` mouseX) `map` \x ->
    if x > 0 then red else blue
```

You never think about “emissions”; you just declare what the color *is*.

Streams (RxJS):

JavaScript

```
mouseX$ = fromEvent(document, 'mousemove').pipe(map(e => e.clientX));
clicks$ = fromEvent(button, 'click');

color$ = mouseX$.pipe(
  switchMap(x => clicks$.pipe(map(() => x))), // time handling is explicit via operators
  map(x => x > 0 ? 'red' : 'blue')
);
```

Here you are orchestrating *dataflow* and using operators to tame the implicit time dimension.

Bottom line

Your intuition is exactly why many people (including Conal Elliott himself) say RxJS is **not** true FRP — it is **reactive programming with streams** (sometimes called “discrete FRP” or just “Rx-style reactivity”).

FRP Thinking puts **time** in the foreground and makes dataflow a consequence.

Streams Thinking puts **dataflow** in the foreground and makes time a background detail you control with operators.

That hidden `[{time, value}]` structure you mentioned is precisely why streams feel “dataflow-first” — the time is there, but the API encourages you to ignore it until you need it. FRP says “let’s make time the star of the show from the beginning.”

This is also why modern signal libraries (Solid, Svelte 5 runes, Vue 3, Angular signals, etc.) feel much closer to original FRP Thinking: they give you that continuous, time-is-first-class feeling without forcing you to write pipelines.

maybe because FRP what first used in Haskell Animations (FRAN) where Time is an essential part of an Animation which you can not neglect

Yes — you’ve nailed the exact historical and conceptual reason.

FRP was literally **born** in the context of **Haskell animations** with **FRAN** (Functional Reactive Animation), the 1997 system by Conal Elliott and Paul Hudak. In that world, **time is not an afterthought** — it is the fundamental dimension you cannot ignore or hide.

Why animations made time first-class

When you animate something (a bouncing ball, a rotating 3D object, a fading color, a moving character), the “value” you care about is its state at every possible moment in time. You don’t want discrete “frames” or “emissions” — you want smooth, continuous change.

In FRAN:

A **Behavior** is defined semantically as a pure function:

$$\text{Behavior } a = \text{Time} \rightarrow a$$

(exactly as you intuited earlier with the hidden `[{time, value}]` idea — but made explicit and continuous).

The entire language is built around composing these time-functions algebraically.

Events are still there, but they mainly *switch* or *sample* behaviors at specific moments.

The FRAN paper states it directly:

“Behaviors are time-varying, reactive values... Most traditional values can be treated as behaviors, and when images are thus treated, they become animations.” [Conal](#)

Conal Elliott has repeatedly emphasized that **continuous time** is one of the two core principles of true FRP (the other being precise denotational semantics). In his talks and writings he says the whole point was to treat animation as “immutable functions of continuous time.” [Conal](#)

That’s why FRP Thinking feels inherently “time-y”

Because it was designed for a domain where neglecting continuous time would break the whole model. You naturally end up asking:

“What is the position/velocity/color as a function of time?”

The runtime samples the behavior at whatever times it needs (for rendering, physics, etc.).

You cannot “hide” time the way streams do, because in animation a discrete approximation (like 60 fps ticks) would feel choppy and would break mathematical composition (e.g., you couldn’t cleanly integrate velocity over time or compose accelerations).

Contrast with Streams (RxJS world)

RxJS and the broader Reactive Extensions lineage came out of **asynchronous event processing and dataflow** (networks, UI events, observables), not animation. So the mental model became:

“Data arrives as discrete values whenever it arrives.”

Time is real but *implicit* (handled by operators like `debounceTime`, `delay`, schedulers).

You never write `Time → Value`; you write pipelines on sequences.

That’s why streams feel like pure dataflow — time is just the medium that carries the data, not the star of the show.

Quick FRAN-style pseudo-example

```
Haskell

-- In FRAN (real Haskell syntax was very close)
ballPosition :: Behavior Point
ballPosition = constant (0,0) `untilB` leftClick `thenB` mousePosition

-- or with physics:
velocity :: Behavior Vector
velocity = integral acceleration

position :: Behavior Point
position = integral velocity
```

Everything is defined as a continuous relationship over time. No “subscribe”, no pipeline of discrete events — just declarative time-functions.

So your feeling is not just intuition — it’s the **origin story** of FRP. FRAN made time inescapable because animations *are* time.

Everything else (modern signals, Rx-style streams, etc.) is a later adaptation or compromise. That’s why classic FRP Thinking still feels more “physics-like” and time-centric than “dataflow-like.”

Spot-on observation! 😊

Look into reactive animation frameworks

Reactive animation frameworks come in two flavors today, and this directly ties back to our discussion on **FRP Thinking** (time-first-class, continuous behaviors/signals) versus **Thinking in Streams** (dataflow with implicit time).

Most “reactive animation” you’ll find in 2026 web development is the **practical, state-driven kind** (declarative + physics-based).

True **classic FRP-style animation** (behaviors as `Time → Value`, algebraic time relationships) lives mostly in Haskell and feels closest to the original FRAN spirit.

1. True FRP / Classic-Style Reactive Animation (Closest to FRAN Thinking)

These treat animation as **declarative relationships over continuous or sampled time** — exactly the “time is first-class” mindset you described. You define *what* a value is as time advances; the framework handles propagation.

Reanimate (Haskell, actively maintained)
The standout modern example. It’s explicitly built around the philosophy:
“Reactive programming is the best paradigm for animating data that changes over time.”
You write **declarative SVG animations** as pure Haskell code. It glues together math viz, physics (built-in Chipmunk2D engine), LaTeX, 3D (via Povray/Blender), and vector graphics. Animations feel like time-varying values evolving naturally — Fourier series, map projections, physics simulations, etc. Very close to FRAN’s continuous-time behaviors, but practical and SVG-first. Inspired by 3Blue1Brown’s Manim but more reactive/functional. [Reanimate.github](#)

Yampa (Haskell)
Arrowized FRP specifically designed for **animation, games, robotics, and simulations**. Uses signal functions (`SF a b`) that model continuous time flow. Supports SDL/OpenGL/HTML DOM. Classic “behaviors + events” model — perfect for physics, game loops, or interactive animations where time is explicit. [Wikipedia](#)

Reflex (Haskell, with reflex-animation / Gloss host)
Modern push-pull FRP. Excellent for **2D graphics and animation** via Gloss (simple 2D drawing) or SDL. You define reactive behaviors and events; it handles continuous animation efficiently. Also works for web DOM. One of the most actively used pure FRP systems today. [Wiki.haskell](#)

Other Haskell mentions (Netwire, reactive-banana, Dunai, etc.) also support animation/graphics but are more general-purpose.
JS equivalents are rare — the closest historical one is **Flapjax** (old behavior/event FRP for the web), but it’s not actively developed. Sodium (cross-language FRP) has JS/TS bindings but is more for general reactive UIs than dedicated animation.

2. Modern “Reactive” Animation Libraries (Practical / Web-Scale)

These dominate React/JS ecosystems in 2026. They feel **reactive** (animations respond to state changes, user input, gestures) but are closer to **signals + physics** than pure streams or classic FRP. Time is still implicit, but the APIs encourage a more “what should this look like right now?” mindset.

Library	Style	Closest to...	Why it feels reactive	Best for
Motion (ex-Framer Motion)	Declarative + motion values	Signals / behaviors	<code>useMotionValue</code> , springs, gestures	UI interactions, React apps
React Spring	Physics-based springs	Continuous time (implicit)	Natural, state-driven motion	Fluid, organic UI animations
GSAP / Anime.js	Timeline / imperative control	Dataflow + timing	Powerful but less “declarative”	Complex sequences, banners
Theatre.js	Code + visual timeline editor	Hybrid reactive	Real-time tweaking of time-varying props	Motion design, cinematic scenes

Motion and **React Spring** are the most “FRP-adjacent” in feel: Motion Values act a lot like signals/behaviors (time-varying values you can derive from), and springs model continuous physics without manual keyframes. They’re declarative and avoid the “pipeline of discrete events” RxJS style. [Motion](#)

Bottom line — tying back to our conversation

If you want **pure FRP Thinking** (time as the star, behaviors as `Time → Value`, algebraic composition): go Haskell → **Reanimate** (most approachable today) or **Yampa/Reflex**. This is the direct descendant of FRAN’s animation-first worldview.

If you want **practical reactive animation in JS/React**: **Motion** or **React Spring**. They give you a lot of the “continuous, time-feeling” reactivity without the full mathematical FRP machinery.

Streams (RxJS) are almost never used for core animation pipelines anymore — they’re too discrete and operational for smooth motion. The industry has shifted toward **physics + signals**.

Reanimate feels like the spiritual successor to FRAN in a usable form: code-first, reactive, time-centric, and beautiful for math/illustrative animations.

Would you like code examples from Reanimate/Yampa, or a deeper dive into how Motion Values compare to classic behaviors? Or shall we look at a specific framework in more detail?

Exactly — RxCSS is a brilliant, pure example of “Thinking in Streams” applied to reactive animations.

David Khourshid (creator of XState) built RxCSS around 2016–2017 as a tiny library that lets you feed RxJS Observables directly into CSS Custom Properties (CSS variables). The browser’s CSS engine then does the rest — handling transitions, animations, transforms, colors, etc. — creating smooth, reactive motion without manual DOM updates or imperative animation loops.

How RxCSS works (Streams-first mindset)

You create observable pipelines (the dataflow), pass them to `RxCSS()`, and it subscribes behind the scenes. Every emission updates a `--custom-property` on the target element (default: `:root` / `<html>`). CSS rules react instantly via `var()`. Classic example from the library (mouse-following reactive ball):

JavaScript

```
// RxJS stream pipeline (pure dataflow)
const mouse$ = Rx.Observable
  .fromEvent(document, 'mousemove')
  .map(e => ({ x: e.clientX, y: e.clientY }))
  .scan(RxCSS.lerp(0.1)); // optional built-in lerp for smooth trailing

// RxCSS does the magic
RxCSS({
  'mouse-x': mouse$.map(m => m.x),
  'mouse-y': mouse$.map(m => m.y)
});
```

Then in CSS (no JavaScript animation code!):

CSS

```
:root {
  --mouse-x: 0;
  --mouse-y: 0;
}

.ball {
  transform:
    translateX(calc(var(--mouse-x) * 1px))
    translateY(calc(var(--mouse-y) * 1px));
  transition: transform 0.01s linear; /* or use CSS @keyframes, springs via custom props, etc. */
}
```

That’s it. The observable pushes values → CSS variables update → browser repaints with hardware-accelerated transforms.

Why this is textbook “Thinking in Streams” (not classic FRP)

Dataflow is front-and-center: You build pipelines with `map`, `scan`, `lerp`, `throttleTime`, `combineLatest`, etc. Time is only present implicitly (via operators or CSS `transition` timing).

No behaviors/signals: There’s no `Time → Value` abstraction. You’re not declaring “what the position is as a continuous function of time.” You’re declaring “here’s a stream of position updates.”

CSS handles the rendering: The library is deliberately minimal — it just wires observables to variables. The “animation” part is offloaded to CSS, which is why it feels so lightweight and performant.

Exactly as you intuited earlier: Each emission is secretly `{ value, implicit-time }`, but the API hides the time dimension behind the stream operators and CSS.

This is the polar opposite of FRAN/Reanimate/Yampa, where you would define a continuous `Behavior` like `position = integral(velocity)` and let the FRP runtime sample it over time. RxCSS is `streams → CSS variables → animation`.

Historical context & legacy

RxCSS came out of David's talks like "Reactive Animations with CSS Variables" (JSConf Iceland 2016) and "Getting Reactive with CSS" (CSSConf EU 2017). It was part of the early wave showing how RxJS + CSS variables could create buttery-smooth, event-driven UIs (mouse followers, scroll parallax, dynamic gradients, drag-with-lerp, etc.) with almost zero imperative code. The library is still on GitHub (415+ stars) and was referenced in many RxJS tutorials and demos (e.g., draggable elements, CSS Husky animations). Today it's no longer actively maintained (modern alternatives like **Motion** / Framer Motion or React Spring do similar things with signals/physics), but it remains one of the cleanest demonstrations of why **Streams Thinking** and CSS variables were such a powerful combo for reactive animation in the browser.

Perfect observation on your part — RxCSS makes the streams → dataflow → implicit-time pattern crystal clear, especially compared to the time-first-class world of original FRP animations. Want a live CodePen-style demo translated to modern RxJS 7+ syntax, or shall we compare it side-by-side with a signal-based equivalent (e.g., Solid/Motion)?